7. 네트워크 통신 시스템

7.1 네트워크 아키텍처 개요

Narwhal의 네트워크 시스템은 Primary와 Worker 노드 간의 효율적인 통신을 담당합니다.

7.1.1 통신 계층 구조

```
Application Layer | Primary ↔ Primary
                  | Worker ↔ Worker
                  | Primary ↔ Worker
──────────────────┼──────────────────
Network Layer     | TCP/UDP Transport
                  | Message Routing
                  | Connection Management
──────────────────┼──────────────────
Physical Layer    | Internet/LAN
```

7.1.2 주요 통신 패턴

- **Broadcast**: 모든 노드에게 메시지 전파
- **Multicast**: 특정 그룹에게 메시지 전송
- **Point-to-Point**: 개별 노드와 직접 통신

- **Request-Response**: 요청-응답 패턴

7.2 메시지 타입과 프로토콜

7.2.1 Primary 메시지 타입

```
#[derive(Serialize, Deserialize, Clone, Debug)]
pub enum PrimaryMessage {
// 헤더 관련
Header(Header),
Vote(Vote),
// 인증서 관련
Certificate(Certificate),
CertificateRequest(CertificateDigest),
CertificateResponse(Certificate),
// 동기화 관련
SyncRequest(SyncRequest),
SyncResponse(SyncResponse),
// 설정 관련
ReconfigureNotification(ReconfigureNotification),
}
```

```
#[derive(Serialize, Deserialize, Clone, Debug)]
pub struct Vote {
```

```
pub header_digest: HeaderDigest,
pub voter: PublicKey,
pub signature: Signature,
}
```

7.2.2 Worker 메시지 타입

```
#[derive(Serialize, Deserialize, Clone, Debug)]
pub enum WorkerMessage {
// 배치 관련
Batch(Batch),
BatchRequest(BatchDigest),
BatchResponse(Batch),
// 트랜잭션 관련
Transaction(Transaction),
TransactionBatch(Vec<Transaction>),
// 상태 관련
WorkerStatus(WorkerStatus),
HealthCheck,
}
```

7.3 네트워크 인터페이스 구현

7.3.1 Primary 네트워크 인터페이스

```
pub struct PrimaryNetwork {
// 연결 관리
connections: HashMap<PublicKey, Connection>,
listener: TcpListener,
// 메시지 큐
outbound_queue: mpsc::Sender<(PublicKey, PrimaryMessage)>,
inbound_queue: mpsc::Receiver<(PublicKey, PrimaryMessage)>,
// 설정
committee: Committee,
network_config: NetworkConfig,
// 메트릭
metrics: Arc<NetworkMetrics>,
}

impl PrimaryNetwork {
pub async fn broadcast_header(&self, header: Header) -> Result<(), NetworkErr
let message = PrimaryMessage::Header(header);
// 모든 다른 Primary에게 전송
for (public_key, _) in &self.committee.authorities {
if *public_key != self.committee.authority(&self.name).unwrap().0 {
self.send_message(public_key.clone(), message.clone()).await?;
```

```
}
}
Ok(())
}
pub  async  fn  send_vote(&self,
target: PublicKey,
vote: Vote
) -> Result<(), NetworkError> {
let  message = PrimaryMessage::Vote(vote);
self.send_message(target, message).await
}
}
```

7.3.2 Worker 네트워크 인터페이스

```
pub  struct WorkerNetwork {
// Primary와의 연결
primary_connection: Connection,
// 다른 Worker들과의 연결
worker_connections: HashMap<WorkerId, Connection>,
// 클라이언트 연결
client_listener: TcpListener,
// 메시지 처리
```

```
message_handler: WorkerMessageHandler,

// 설정

worker_config: WorkerConfig,

}



impl WorkerNetwork {

pub  async  fn  broadcast_batch(&self, batch: Batch) -> Result<(), NetworkError>

let  message = WorkerMessage::Batch(batch);

// 모든 다른 Worker에게 전송

for (worker_id, connection) in &self.worker_connections {

connection.send(message.clone()).await?;

}

Ok(())

}

pub  async  fn  send_to_primary(&self,

message: WorkerToPrimaryMessage

) -> Result<(), NetworkError> {

self.primary_connection.send(message).await

}

}
```

7.4 연결 관리 시스템

7.4.1 연결 풀 관리

```
pub struct ConnectionPool {
active_connections: HashMap<SocketAddr, Arc<Connection>>,
connection_config: ConnectionConfig,
max_connections: usize,
connection_timeout: Duration,
}

impl ConnectionPool {
pub async fn get_connection(&mut self,
addr: SocketAddr
) -> Result<Arc<Connection>, NetworkError> {
// 기존 연결 재사용
if let Some(conn) = self.active_connections.get(&addr) {
if conn.is_healthy().await {
return Ok(conn.clone());
} else {
self.active_connections.remove(&addr);
}
}
// 새 연결 생성
let connection = self.create_connection(addr).await?;
```

```
let arc_conn = Arc::new(connection);
self.active_connections.insert(addr, arc_conn.clone());
Ok(arc_conn)
}
async fn create_connection(&self, addr: SocketAddr) -> Result<Connection, Netw
let stream = timeout(
self.connection_timeout,
TcpStream::connect(addr)
).await??;
Ok(Connection::new(stream, self.connection_config.clone()))
}
}
```

7.4.2 연결 상태 모니터링

```
pub struct ConnectionMonitor {
health_check_interval: Duration,
max_retry_attempts: usize,
backoff_strategy: BackoffStrategy,
}

impl ConnectionMonitor {
pub async fn monitor_connections(&mut self,
```

```
pool: &mut ConnectionPool
) {
let mut interval = tokio::time::interval(self.health_check_interval);
loop {
interval.tick().await;
// 모든 연결 상태 확인
let mut failed_connections = Vec::new();
for (addr, connection) in &pool.active_connections {
if !connection.is_healthy().await {
failed_connections.push(*addr);
}
}
// 실패한 연결 재시도
for addr in failed_connections {
self.retry_connection(pool, addr).await;
}
}
}
}
```

7.5 메시지 직렬화와 압축

7.5.1 직렬화 시스템

```rust
pub trait MessageCodec {
type Message;
type Error;
fn encode(&self, message: &Self::Message) -> Result<Vec<u8>, Self::Error>;
fn decode(&self, data: &[u8]) -> Result<Self::Message, Self::Error>;
}

pub struct BincodeCodec;

impl MessageCodec for BincodeCodec {
type Message = PrimaryMessage;
type Error = bincode::Error;
fn encode(&self, message: &Self::Message) -> Result<Vec<u8>, Self::Error> {
bincode::serialize(message)
}
fn decode(&self, data: &[u8]) -> Result<Self::Message, Self::Error> {
bincode::deserialize(data)
}
}
```

7.5.2 압축 최적화

```rust
pub struct CompressionLayer {
```

```
compression_threshold: usize,

compression_algorithm: CompressionAlgorithm,

}



impl CompressionLayer {

pub  fn  compress_if_beneficial(&self, data: &[u8]) -> Vec<u8> {

if  data.len() > self.compression_threshold {

match  self.compression_algorithm {

CompressionAlgorithm::Gzip => {

self.gzip_compress(data)

}

CompressionAlgorithm::Lz4 => {

self.lz4_compress(data)

}

CompressionAlgorithm::Zstd => {

self.zstd_compress(data)

}

}

} else {

data.to_vec()

}

}
```

```
}
```

7.6 네트워크 보안

7.6.1 TLS 암호화

```
pub struct SecureConnection {
tls_stream: TlsStream<TcpStream>,
peer_certificate: Certificate,
encryption_config: EncryptionConfig,
}

impl SecureConnection {
pub async fn establish_secure_connection(
addr: SocketAddr,
tls_config: TlsConfig,
) -> Result<Self, SecurityError> {
let tcp_stream = TcpStream::connect(addr).await?;
let connector = TlsConnector::from(tls_config.client_config);
let tls_stream = connector.connect("narwhal-node", tcp_stream).await?;
let peer_cert = tls_stream.peer_certificate()
.ok_or(SecurityError::NoPeerCertificate)?;
Ok(SecureConnection {
```

```
tls_stream,
peer_certificate: peer_cert,
encryption_config: tls_config.encryption_config,
})
}
}
```

7.6.2 메시지 인증

```
pub struct MessageAuthenticator {
signing_key: PrivateKey,
verification_keys: HashMap<PublicKey, PublicKey>,
}

impl MessageAuthenticator {
pub fn sign_message(&self, message: &[u8]) -> Signature {
self.signing_key.sign(message)
}
pub fn verify_message(&self,
message: &[u8],
signature: &Signature,
sender: &PublicKey
) -> bool {
```

```
if let Some(public_key) = self.verification_keys.get(sender) {
public_key.verify(message, signature).is_ok()
} else {
false
}
}
}
```

7.7 네트워크 최적화

7.7.1 배치 전송

```
pub struct BatchSender {
batch_size: usize,
batch_timeout: Duration,
pending_messages: Vec<PrimaryMessage>,
last_send_time: Instant,
}

impl BatchSender {
pub async fn send_message(&mut self,
message: PrimaryMessage,
connection: &Connection
```

```
) -> Result<(), NetworkError> {
self.pending_messages.push(message);
// 배치 조건 확인
if self.should_send_batch() {
self.flush_batch(connection).await?;
}
Ok(())
}
fn should_send_batch(&self) -> bool {
self.pending_messages.len() >= self.batch_size ||
self.last_send_time.elapsed() >= self.batch_timeout
}
async fn flush_batch(&mut self,
connection: &Connection
) -> Result<(), NetworkError> {
if !self.pending_messages.is_empty() {
let batch = MessageBatch {
messages: std::mem::take(&mut self.pending_messages),
timestamp: SystemTime::now(),
};
connection.send_batch(batch).await?;
self.last_send_time = Instant::now();
}
```

```
Ok(())
}
}
```

7.7.2 적응적 대역폭 제어

```
pub struct BandwidthController {
current_bandwidth: u64,
target_bandwidth: u64,
congestion_window: u32,
rtt_estimator: RttEstimator,
}

impl BandwidthController {
pub fn adjust_sending_rate(&mut self, network_feedback: NetworkFeedback) {
match network_feedback {
NetworkFeedback::Congestion => {
self.congestion_window = self.congestion_window / 2;
}
NetworkFeedback::Normal => {
self.congestion_window += 1;
}
NetworkFeedback::FastPath => {
```

```
self.congestion_window = (self.congestion_window * 1.2) as u32;
}
}
self.current_bandwidth = self.calculate_bandwidth();
}
}
```

7.8 장애 처리와 복구

7.8.1 네트워크 분할 처리

```
pub struct PartitionHandler {
partition_detector: PartitionDetector,
recovery_strategy: RecoveryStrategy,
isolation_timeout: Duration,
}

impl PartitionHandler {
pub async fn handle_network_partition(&mut self) -> Result<(), PartitionErro
// 분할 감지
if self.partition_detector.is_partitioned().await {
// 격리 모드 진입
self.enter_isolation_mode().await?;
```

```
// 복구 대기
self.wait_for_recovery().await?;
// 재연결 시도
self.attempt_reconnection().await?;
}
Ok(())
}
}
```

7.8.2 메시지 재전송 메커니즘

```
pub  struct ReliableDelivery {
pending_messages: HashMap<MessageId, PendingMessage>,
retry_policy: RetryPolicy,
acknowledgment_timeout: Duration,
}

struct PendingMessage {
message: PrimaryMessage,
send_time: Instant,
retry_count: usize,
target: PublicKey,
}
```

```
impl ReliableDelivery {
pub  async  fn  send_reliable(&mut  self,
message: PrimaryMessage,
target: PublicKey
) -> Result<(), DeliveryError> {
let  message_id = MessageId::new();
let  pending = PendingMessage {
message: message.clone(),
send_time: Instant::now(),
retry_count: 0,
target: target.clone(),
};
self.pending_messages.insert(message_id, pending);
self.send_with_retry(message_id).await
}
}
```

이러한 네트워크 시스템을 통해 Narwhal은 높은 성능과 안정성을 가진 분산 컨센서스를 구현할 수 있습니다.